



ABLESTACK Online Docs  
ABLESTACK-V4.0-4.0.15

# Event 관리

# Event 관리

이벤트는 클라우드 환경과 관련된 가상 및 물리적 리소스의 상태에서 중요하거나 의미 있는 변화를 말합니다. 이벤트는 모니터링 시스템, 사용량 및 청구 시스템 또는 기타 이벤트 기반 워크플로 시스템에서 패턴을 식별하고 올바른 비즈니스 결정을 내리는 데 사용됩니다. Mold 에서 이벤트는 가상 또는 물리적 리소스의 상태 변경, 사용자가 수행한 작업(작업 이벤트) 또는 정책 기반 이벤트(경고)를 뜻합니다.

## Event 로그

Mold 이벤트 로그에는 두 가지 유형의 이벤트가 기록됩니다. 먼저 표준 이벤트는 이벤트의 성공 또는 실패를 기록하고 실패한 작업 또는 프로세스를 식별하는 데 사용할 수 있습니다. 또한 장기 실행 작업 이벤트인 비동기 작업 이벤트는 작업이 예약될 때, 시작될 때 및 완료될 때 기록됩니다. 그리고 기타 장기 실행 동기 작업은 작업이 시작되고 완료될 때를 기록합니다. 장기 실행 동기 및 비동기 이벤트 로그를 사용하여 보류 중인 작업의 상태에 대한 자세한 정보를 얻고 중단되거나 시작되지 않은 작업을 식별할 수 있습니다. 다음 섹션에서는 이러한 이벤트에 대한 자세한 정보를 제공합니다.

## 알림

이벤트 알림 프레임 워크는 Management Server 구성 요소가 Mold 이벤트를 게시하고 구독할 수 있는 수단을 제공합니다. 이벤트 알림은 Management Server에서 이벤트 버스 추상화 개념을 구현하여 이루어집니다.

상태 변경에 대한 새로운 이벤트인 리소스 상태 변경이 이벤트 알림 프레임 워크의 일부로 도입되었습니다. 사용자 VM, 볼륨, NIC, 네트워크, Public IP, 스냅샷 및 템플릿과 같은 모든 리소스는 상태 시스템과 연결되며 상태 변경의 일부로 이벤트를 생성합니다. 이는 리소스 상태의 변경으로 인해 상태 변경 이벤트가 발생하고 이벤트가 이벤트 버스의 해당 상태 머신에 게시됨을 의미합니다. 모든 Mold 이벤트 (경고, 작업 이벤트, 사용 이벤트) 및 리소스 상태 변경 이벤트의 추가 범주가 이벤트 버스에 게시됩니다.

## 구현

Mold 구성 요소 및 확장 플러그인이 AMQP (Advanced Message Queuing Protocol) 클라이언트를 사용하여 이벤트를 구독할 수 있도록 하는 이벤트 버스가 Management Server에 도입되었습니다. 기본적으로 Mold에서 이벤트 버스의 구현은 RabbitMQ AMQP 클라이언트를 사용하는 플러그인으로 제공되고 있습니다. AMQP 클라이언트는 게시된 이벤트들을 호환 가능한 AMQP 서버로 푸시합니다. 따라서 모든 Mold 이벤트는 AMQP 서버의 거래소에 게시됩니다.

추가적으로 In-memory 구현과 Apache Kafka 구현도 모두 사용할 수 있습니다.

## 사용 사례

다음은 몇 가지 사용 사례입니다.

- 사용량 또는 청구 엔진 : 타사 클라우드 사용량 솔루션은 Mold에 연결하여 Mold 이벤트를 구독하고 사용량 데이터를 생성할 수 있는 플러그인을 구현할 수 있습니다. 사용(usage) 데이터는 사용(usage) 소프트웨어에서 사용됩니다.
- AMQP 플러그인은 메시지 대기열에 모든 이벤트를 배치할 수 있으며 AMQP 메시지 브로커는 구독자에게 주제 기반 알림을 제공할 수 있습니다.

- 게시 및 구독 알림 서비스는 주제 기반 구독 및 알림과 같은 이벤트 알림을 위한 풍부한 API 세트를 제공하는 Mold의 플러그 형 서비스로 구현 될 수 있습니다. 또한 플러그 형 서비스는 멀티 테넌시, 인증 및 권한 부여 문제를 처리할 수 있습니다.

## AMQP 구성

Mold 관리자는 다음 일회성 구성을 수행하여 이벤트 알림 프레임 워크를 활성화합니다. 런타임에는 변경 사항이 동작을 제어할 수 없습니다.

1. /etc/cloudstack/management/META-INF/cloudstack/core 폴더를 만듭니다.
2. 해당 폴더 내에서 spring-event-bus-context.xml 을 엽니다.
3. 다음과 같이 eventNotificationBus 라는 Bean을 정의하십시오.
  - name : 빈의 이름을 지정합니다.
  - server : RabbitMQ AMQP 서버의 이름 또는 IP 주소입니다.
  - port : RabbitMQ 서버가 실행되는 포트입니다.
  - username : RabbitMQ 서버에 액세스하기 위한 계정과 관련된 사용자 이름입니다.
  - password : RabbitMQ 서버에 액세스하기 위한 계정의 사용자 이름과 관련된 암호입니다.
  - exchange : Mold 이벤트가 게시되는 RabbitMQ 서버의 교환 이름입니다.

샘플 bean은 다음과 같습니다.

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:context="http://www.springframework.org/schema/context"
  xmlns:aop="http://www.springframework.org/schema/aop"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
    http://www.springframework.org/schema/aop
    http://www.springframework.org/schema/aop/spring-aop-3.0.xsd
    http://www.springframework.org/schema/context
    http://www.springframework.org/schema/context/spring-context-3.0.xsd">
  <bean id="eventNotificationBus"
    class="org.apache.cloudstack.mom.rabbitmq.RabbitMQEventBus">
    <property name="name" value="eventNotificationBus"/>
    <property name="server" value="127.0.0.1"/>
    <property name="port" value="5672"/>
    <property name="username" value="guest"/>
    <property name="password" value="guest"/>
    <property name="exchange" value="cloudstack-events"/>
  </bean>
</beans>
```

eventNotificationBus bean은 org.apache.cloudstack.mom.rabbitmq.RabbitMQEventBus 클래스를 대표합니다. 사용자 이름과 암호에 암호화된 값을 사용하려면 자격 증명 파일의 변수로 전달하기 위해 Bean을 포함해야 합니다. 아래에 샘플이 제공됩니다.

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop-3.0.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-3.0.xsd"
>

    <bean id="eventNotificationBus"
class="org.apache.cloudstack.mom.rabbitmq.RabbitMQEventBus">
        <property name="name" value="eventNotificationBus"/>
        <property name="server" value="127.0.0.1"/>
        <property name="port" value="5672"/>
        <property name="username" value="${username}"/>
        <property name="password" value="${password}"/>
        <property name="exchange" value="cloudstack-events"/>
    </bean>

    <bean id="environmentVariablesConfiguration"
class="org.jasypt.encryption.pbe.config.EnvironmentStringPBEConfig">
        <property name="algorithm" value="PBEWithMD5AndDES" />
        <property name="passwordEnvName" value="APP_ENCRYPTION_PASSWORD" />
    </bean>

    <bean id="configurationEncryptor"
class="org.jasypt.encryption.pbe.StandardPBEStringEncryptor">
        <property name="config" ref="environmentVariablesConfiguration" />
    </bean>

    <bean id="propertyConfigurer"
class="org.jasypt.spring3.properties.EncryptablePropertyPlaceholderConfigurer">
        <constructor-arg ref="configurationEncryptor" />
        <property name="location" value="classpath:/cred.properties" />
    </bean>
</beans>

```

cred.properties 라는 동일한 이름의 폴더에 새 파일을 만들고 사용자 이름 및 암호 값을 jasypt 암호화 문자열로 지정합니다.

샘플 : 두 필드의 값으로 guest 가 있는 샘플

```

username=nh2XrM7jWHMG4VQK18iiBQ==
password=nh2XrM7jWHMG4VQK18iiBQ==

```

4. 관리 서버를 다시 시작하십시오.

## Kafka 구성

Mold 관리자는 다음 일회성 구성을 수행하여 이벤트 알림 프레임 워크를 활성화합니다. 런타임에는 변경 사항이 동작을 제어할 수 없습니다.

1. 유효한 kafka 구성 속성을 포함하는 /etc/cloudstack/management/kafka.producer.properties에 적절한 구성 파일을 만듭니다. properties에는 추가 topic이 포함될 수 있습니다. 특정 값이 제공되지 않는 경우에는 기본적으로

cloudstack 이 사용됩니다. 일반적으로 올바르게 시작하려면 key.serializer 및 value.serializer 가 필요하지만 생략 될 수 있으며 기본값은 org.apache.kafka.common.serialization.StringSerializer 입니다.

2. /etc/cloudstack/management/META-INF/cloudstack/core 폴더를 만듭니다.

3. 해당 폴더 내에서 spring-event-bus-context.xml 을 엽니다.

4. 단일 이름 속성으로 eventNotificationBus 라는 이름의 bean을 정의합니다. 샘플 bean은 다음과 같습니다.

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xmlns:context="http://www.springframework.org/schema/context"
        xmlns:aop="http://www.springframework.org/schema/aop"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
                            http://www.springframework.org/schema/beans/spring-beans-3.0.xsd
                            http://www.springframework.org/schema/aop
                            http://www.springframework.org/schema/aop/spring-aop-3.0.xsd
                            http://www.springframework.org/schema/context
                            http://www.springframework.org/schema/context/spring-context-3.0.xsd">
    <bean id="eventNotificationBus" class="org.apache.cloudstack.mom.kafka.KafkaEventBus">
        <property name="name" value="eventNotificationBus"/>
    </bean>
</beans>
```

5. 관리 서버를 다시 시작하십시오.

## 표준 이벤트

이벤트 로그는 세 가지 유형의 표준 이벤트를 기록합니다.

- INFO. 이 이벤트는 작업이 성공적으로 수행되었을 때 생성됩니다.
- WARN. 이 이벤트는 다음과 같은 상황에서 생성됩니다.
  - 템플릿 다운로드 모니터링 중 네트워크 연결이 끊어진 경우
  - 템플릿 다운로드가 중단 된 경우.
  - 스토리지 서버의 문제로 인해 볼륨이 미리 스토리지 서버로 장애 조치되는 경우.
- ERROR. 이 이벤트는 작업이 성공적으로 수행되지 않은 경우 생성됩니다.

## 장시간 실행 작업 이벤트

이벤트 로그는 세 가지 유형의 표준 이벤트를 기록합니다.

- INFO. 이 이벤트는 작업이 성공적으로 수행되었을 때 생성됩니다.
- WARN. 이 이벤트는 다음과 같은 상황에서 생성됩니다.
  - 템플릿 다운로드 모니터링 중 네트워크 연결이 끊어진 경우
  - 템플릿 다운로드가 중단 된 경우.
  - 스토리지 서버의 문제로 인해 볼륨이 미리 스토리지 서버로 장애 조치되는 경우.
- ERROR. 이 이벤트는 작업이 성공적으로 수행되지 않은 경우 생성됩니다.

## 이벤트 로그 쿼리

데이터베이스 로그는 사용자 인터페이스에서 쿼리할 수 있습니다. 시스템에서 캡처한 이벤트 목록에는 다음이 포함됩니다.

- 가상 머신 생성, 삭제 및 지속적인 관리 작업
- 가상 라우터 생성, 삭제 및 지속적인 관리 작업
- 템플릿 생성 및 삭제
- 네트워크 / 로드 밸런서 규칙 생성 및 삭제
- 스토리지 볼륨 생성 및 삭제
- 사용자 로그인 및 로그 아웃

## 이벤트 및 경고 삭제 및 보관

Mold는 더 이상 구현하지 않을 기존 경고 및 이벤트를 삭제하거나 보관할 수 있는 기능을 제공합니다. 데이터베이스에서 해결할 수 없거나 해결하지 않을 경고 또는 이벤트를 정기적으로 삭제하거나 보관할 수 있습니다.

세부 정보 페이지를 사용하여:

- 개별 경고 또는 이벤트를 삭제하거나 보관할 수 있습니다.
- 여러 경고 또는 이벤트를 동시에 삭제하려면 해당 컨텍스트 메뉴를 사용할 수 있습니다.
- 일정 기간 동안 범주별로 경고 또는 이벤트를 삭제할 수 있습니다. 예를 들어 USER.LOGOUT , VM.DESTROY , VM.AG.UPDATE , CONFIGURATION.VALUE.EDI 등과 같은 범주를 선택할 수 있습니다 .
- 보관되거나 삭제 된 이벤트 또는 경고 수를 볼 수 있습니다.

삭제 또는 아카이브 알림을 지원하기 위해 다음 글로벌 매개 변수가 추가되었습니다.

- alert.purge.delay : 지정된 일 수보다 오래된 경고가 제거됩니다. 알림을 자동으로 제거하지 않으려면 값을 0으로 설정하십시오.
- alert.purge.interval : 경고 제거 스레드를 실행하기 전에 대기하는 간격 (초)입니다. 기본값은 86400 초 (1 일)입니다.

### Note

보관된 경고 또는 이벤트는 UI에서 또는 API를 사용하여 볼 수 없습니다. 감사 또는 규정 준수 목적으로 데이터베이스에 유지됩니다.

## 권한

다음은 고려하세요:

- 루트 관리자는 하나 이상의 경고 또는 이벤트를 삭제하거나 보관할 수 있습니다.
- 도메인 관리자 또는 최종 사용자는 하나 이상의 이벤트를 삭제하거나 보관할 수 있습니다.

## 순서

1. Mold UI에 관리자로 로그인하십시오.
2. 왼쪽 메뉴 메뉴에서 이벤트를 클릭합니다.
3. 다음 중 하나를 수행하십시오.
  - 이벤트를 아카이브하려면 이벤트를 선택한 후 "이벤트 아카이브"를 클릭하여 보관하십시오.
  - 이벤트를 삭제하려면 "이벤트 삭제"를 클릭하십시오.
4. 확인을 클릭하십시오.

# ABLESTACK Online Docs